

メイン指示書

このプロジェクトのセットアップに関する詳細な指示は、以下の各セクションを参照してください。

- [要件定義](#)
- [エージェント構成](#)
- [ガードレール](#)
- [完了報告](#)

プロジェクト初期セットアップ指示書 (Claude Code 向け)

あなたはこのリポジトリの初期セットアップ担当です。以下を順に実施し、最後に「何を作成・変更したか」をまとめて報告してください。
不明点は推測で進めず、着手前に質問してください。

要件定義

1. プロジェクトの目的と開発環境

- 目的:
《障害福祉事業所のシステム構築》
- 主要技術:
《Appsheet / Salesforce / GCP SQL / スプシ / GAS》
- 実行環境:
《OS(クロムブラウザI)》
- 上記を `CLAUDE.md` の冒頭に「プロジェクトコンテキスト」として記載すること。
- Windows + PowerShell 前提の場合、シェルスクリプトで POSIX 専用構文 (`/dev/null`, `\$VAR` 等) を使わないこと。

2. ECC (Everything Claude Code) から最適なスキルセットを用意

<https://github.com/affaan-m/ECC.git>

- このプロジェクトの技術スタックに合致する ****ECCスキルのみ**** を選定し、プロジェクトスコープ(`.claude/`)に導入する。
- 関係のないスキルは入れない(起動時の読み込みと判断ノイズを減らすため)。
- 選定理由を 1 行ずつ添えて、導入リストを報告すること。

3. エージェントチーム構成(モデルは Opus)

エージェント構成

- ``.claude/agents/`` に 3 役を Opus ベースで構成する:
 - ``planner`` ... 計画立案 (機能 / データモデル / UI・UX) → ``docs/plans/*.md``
 - ``implementer`` ... 計画書に基づく実装 (コード)
 - ``verifier`` ... 品質検証 (テスト / セキュリティ / 設計 / コスト)。コードは修正せずレビューのみ。
必ずクロス検証で実施。
- 各エージェントは ``.claude/agent-memory/<name>/MEMORY.md`` を持ち、横断知識を蓄積する。
- ワークフロー: 要件提示 → planner → 承認 → implementer → verifier → 必要なら差し戻し。

ガードレール

4. Claude Code の自律判断・独自実装を履歴として保持

- あなた (Claude Code) が ****指示に無い判断・設計選択・実装**** を行った場合は、必ず ``docs/decisions/DECISIONS.md`` に追記する。
- 1 エントリの形式:
 - ``日付` / `判断の要約` / `なぜそうしたか(理由)` / `代替案と却下理由` / `影響範囲(ファイル)``
- 後から人間が「なぜこうなっているか」を追えることを最優先。勝手に過去エントリを書き換えない(追記のみ)。

5. CLI 認証のプロジェクト専用コンテキスト分離 (md + hook で必ずルール化)

他プロジェクトのアカウント／プロジェクトと ****競合・混同しない**** よう、二重に縛る:

- ****Single Source of Truth****: ``.claude/project-context.env`` に期待する CLI コンテキスト (例: `gcloud config 名 / account / project / region`、`firebase project`、`gh アカウント等`) を定義する。
- ****CLAUDE.md にルール化****: 「このリポジトリの CLI は必ず専用 context で動かす」旨と、期待値テーブル、ドリフト時の復旧コマンドを明記する。
- ****hook で自動ガード**** (``.claude/hooks/``):
 - ``SessionStart``: 専用 context を幕等に `activate` (後述のとおり ****ノンブロッキング****で)。
 - ``PreToolUse`` (Bash / PowerShell): ``gcloud` / `gsutil` / `bq` / `firebase` / `gh`` 実行直前に context を検証し、ドリフト時は `exit 2` で ****block + 復旧コマンドを stderr 出力****。``.--project=`` 等の明示フラグが期待値以外を指す場合も block。
 - 期待値は ``.claude/project-context.env`` を ``source`` して参照し、ハードコードしない。
- 一時無効化の逃げ道として ``.claude/.skip-*-check`` マーカー方式を用意し、``.gitignore`` に登録 (commit 禁止)。

6. 起動パフォーマンスのハードニング (必須・最初に整える)

Cursor 等の統合ターミナルで Claude Code が「起動時に立ち上がらない／時間をおくと読まれる」

問題を防ぐため、起動時の初期化を軽くする:

- ****MCP は棚卸しして必要分のみ****:
 - グローバル (`~/.claude.json``) に多数の MCP が登録されていても、このプロジェクトで使うものだけをプロジェクトスコープ (``.mcp.json``) で有効化する方針にする。
 - 不要な MCP の一斉起動が起動遅延の最大要因になるため、使わないものは起動させない。
- ****SessionStart フックは軽量・ノンブロッキング****:
 - 外部 CLI (``gcloud`` 等) の呼び出し回数を最小化する (幕等な ``activate`` 1 回に集約し、防御的な ``set` / `get-value`` の連打を避ける)。Windows では 1 呼び出しごとにプロセス起動コストが大きい。
 - 可能ならバックグラウンド実行 (``&``) でセッション起動をブロックしない。

実ガードは PreToolUse 側が担うため安全性は保てる。

- ****rules / ドキュメントの重複排除****:
 - 同名・同内容の rules を二重に読み込ませない(読み込み量と処理時間の無駄を排除)。
 - `CLAUDE.md` は肥大化させず、参照ドキュメントへリンクで逃がす。
- ****ターミナル自動復元に依存しない運用を明記****:
 - Cursor 起動と同時の自動復元はレースを生むため、
「ワークスペースのインデックス／言語サーバーが落ち着いてから手動で `claude` 起動」を
README/CLAUDE.md に記す。
 - 参考設定: `"terminal.integrated.enablePersistentSessions": false``

完了報告

7. 完了報告

セットアップ後、以下を箇条書きで報告すること:

- 作成・変更したファイル一覧(`CLAUDE.md` / `.claude/agents/\*` / `.claude/hooks/\*` /
`.claude/project-context.env` / `docs/decisions/DECISIONS.md` / `.mcp.json` 等)
- 導入した ECC スキルとその理由
- 起動ハードニングで実際に削った／軽くした項目
- 人間が次に手で確認・入力すべき事項(例: `project-context.env` の値埋め)